39.3 多 Agent 沙箱工具

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~310k tokens，输出 ~5k tokens（本节）

在多 Agent 架构中，不同 Agent 的能力和权限应当有所区分。一个负责代码执行的 Agent 需要文件读写和 shell 工具，但不应该有权限操控浏览器或发送消息到通道。OpenClaw 通过**沙箱工具策略**（Sandbox Tool Policy）实现了这种精细的工具级隔离。

> **衍生解释：最小权限原则**
> 最小权限原则（Principle of Least Privilege）是计算机安全的基本原则之一。它要求每个模块只拥有完成其任务所必需的最小权限集合。在 AI Agent 场景中，这意味着子 Agent 不应该获得超出其任务所需的工具访问权限——一个只需要读文件的 Agent 不应该拥有执行 shell 命令的能力。

39.3.1 跨 Agent 工具隔离

默认工具策略

`sandbox/constants.ts` 定义了沙箱环境中的工具白名单和黑名单：

```typescript
// src/agents/sandbox/constants.ts

export const DEFAULT_TOOL_ALLOW = [
  "exec",              // Shell 命令执行
  "process",           // 进程管理
  "read",              // 文件读取
  "write",             // 文件写入
  "edit",              // 文件编辑
  "apply_patch",       // 补丁应用
  "image",             // 图像处理
  "sessions_list",     // 列出会话
  "sessions_history",  // 查看会话历史
  "sessions_send",     // 向会话发送消息
  "sessions_spawn",    // 派生子 Agent
  "session_status",    // 查看会话状态
] as const;

export const DEFAULT_TOOL_DENY = [
  "browser",           // 浏览器自动化
  "canvas",            // Canvas/A2UI
  "nodes",             // 设备节点控制
  "cron",              // 定时任务
  "gateway",           // Gateway 管理
  ...CHANNEL_IDS,      // 所有消息通道 (telegram, discord, slack...)
] as const;
```

白名单设计体现了"**执行能力开放，通信能力封闭**"的安全哲学：

类别	工具	策略	理由
代码执行	exec, process	允许	沙箱已提供进程级隔离
文件操作	read, write, edit, apply_patch	允许	沙箱限制了文件系统范围
多模态	image	允许	图像处理是常见需求
会话管理	sessions_*	允许	子 Agent 需要与其他Agent 协作
浏览器	browser	禁止	浏览器有独立的沙箱方案
UI 渲染	canvas	禁止	子 Agent 不应直接操作 UI
设备控制	nodes	禁止	设备操作需要显式授权

类别	工具	策略	理由
调度	cron	禁止	子 Agent 是临时的，不应创建持久任务
消息通道	telegram, discord...	禁止	防止子 Agent 擅自发送消息

注意 `CHANNEL_IDS` 是从通道注册表动态展开的，这意味着每个已注册的消息通道都会自动被加入黑名单——即使未来添加新通道也不需要手动更新。

工具策略解析

`tool-policy.ts` 实现了三级优先级的策略解析：

```typescript
// src/agents/sandbox/tool-policy.ts

export function resolveSandboxToolPolicyForAgent(
  cfg?: OpenClawConfig,
  agentId?: string,
): SandboxToolPolicyResolved {
  const agentConfig = cfg && agentId
    ? resolveAgentConfig(cfg, agentId)
    : undefined;

  // Agent 级配置 > 全局配置 > 默认常量
  const agentAllow = agentConfig?.tools?.sandbox?.tools?.allow;
  const agentDeny  = agentConfig?.tools?.sandbox?.tools?.deny;
  const globalAllow = cfg?.tools?.sandbox?.tools?.allow;
  const globalDeny  = cfg?.tools?.sandbox?.tools?.deny;

  const deny = Array.isArray(agentDeny) ? agentDeny
    : Array.isArray(globalDeny) ? globalDeny
    : [...DEFAULT_TOOL_DENY];

  const allow = Array.isArray(agentAllow) ? agentAllow
    : Array.isArray(globalAllow) ? globalAllow
    : [...DEFAULT_TOOL_ALLOW];

  // 展开工具组别名
  const expandedDeny = expandToolGroups(deny);
  let expandedAllow = expandToolGroups(allow);

  // image 工具的特殊处理：除非显式禁止，否则始终允许
  if (
    !expandedDeny.includes("image") &&
    !expandedAllow.includes("image")
  ) {
    expandedAllow = [...expandedAllow, "image"];
  }

  return { allow: expandedAllow, deny: expandedDeny, sources: { ... } };
}
```

策略解析的优先级链：

```
Agent 级策略（agents.list[].tools.sandbox.tools）
    ↓（未设置）
全局策略（tools.sandbox.tools）
    ↓（未设置）
默认常量（DEFAULT_TOOL_ALLOW / DEFAULT_TOOL_DENY）
```

返回的 `SandboxToolPolicyResolved` 中包含 `sources` 字段，记录了每条策略的来源（`"agent"` / `"global"` / `"default"`），便于调试和审计。

`image` 工具享有特殊待遇：除非在黑名单中被显式禁止，否则即使白名单中未列出，也会被自动添加。这是因为多模态交互（如截图分析、图像生成）已经成为 AI Agent 的基础能力。

工具名称匹配

工具名称的匹配支持精确匹配和通配符模式：

```typescript
type CompiledPattern =
  | { kind: "all" }           // "*" — 匹配所有
  | { kind: "exact"; value: string }  // 精确匹配
  | { kind: "regex"; value: RegExp }; // 通配符转正则

function compilePattern(pattern: string): CompiledPattern {
  const normalized = pattern.trim().toLowerCase();
  if (normalized === "*") return { kind: "all" };
  if (!normalized.includes("*")) {
    return { kind: "exact", value: normalized };
  }
  // "sessions_*" → /^sessions_.*$/
  const escaped = normalized
    .replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
  return {
    kind: "regex",
    value: new RegExp(`^${escaped.replaceAll("\\*", ".*")}$`),
  };
}

export function isToolAllowed(
  policy: SandboxToolPolicy,
  name: string,
) {
  const normalized = name.trim().toLowerCase();
  // 黑名单优先：命中 deny 则拒绝
  if (matchesAny(normalized, compilePatterns(policy.deny))) {
    return false;
  }
  // 白名单为空则全部允许
  const allow = compilePatterns(policy.allow);
  if (allow.length === 0) return true;
  // 必须命中白名单
  return matchesAny(normalized, allow);
}
```

匹配算法遵循**黑名单优先**原则：

1. 如果工具名命中黑名单 → 拒绝（无论白名单如何）
2. 白名单为空 → 允许所有未被黑名单拒绝的工具
3. 白名单非空 → 必须命中白名单才允许

通配符 `*` 被转换为正则表达式中的 `.*` 。例如， `"sessions_*"` 会匹配 `sessions_list` 、 `sessions_spawn` 等所有以 `sessions_` 开头的工具。

39.3.2 沙箱Agent 配置

沙箱模式与范围

每个 Agent 的沙箱配置通过 `resolveSandboxConfigForAgent()` 解析：

```
// src/agents/sandbox/config.ts

export function resolveSandboxConfigForAgent(
  cfg?: OpenClawConfig,
  agentId?: string,
): SandboxConfig {
  const globalSandbox = cfg?.agents?.defaults?.sandbox;
  const agentSandbox = agentConfig?.sandbox;

  return {
    mode: agentSandbox?.mode ?? globalSandbox?.mode ?? "off",
    scope: resolveSandboxScope({ ... }),
    workspaceAccess: ... ?? "none",
    workspaceRoot: ... ?? DEFAULT_SANDBOX_WORKSPACE_ROOT,
    docker: resolveSandboxDockerConfig({ ... }),
    browser: resolveSandboxBrowserConfig({ ... }),
    tools: { allow: ..., deny: ... },
    prune: resolveSandboxPruneConfig({ ... }),
  };
}
```

沙箱配置的关键维度：

维度	可选值	说明
`mode`	`"off"` / `"auto"` / `"always"`	沙箱启用模式
`scope`	`"agent"` / `"session"` / `"shared"`	容器共享粒度
`workspaceAccess`	`"none"` / `"readonly"` / `"readwrite"`	宿主工作空间挂载权限

`scope` 决定了 Docker 容器的生命周期：

- **"agent"**：每个 Agent 共享一个容器。容器随 Agent 的首次使用创建，空闲后回收。
- **"session"**：每个会话创建独立容器。最高隔离度，但资源消耗最大。
- **"shared"**：所有 Agent 共享一个全局容器。资源消耗最低，但无隔离。

Docker 容器配置

`resolveSandboxDockerConfig()` 解析 Docker 容器的安全参数：

```
export function resolveSandboxDockerConfig(params: {
  scope: SandboxScope;
  globalDocker?: Partial<SandboxDockerConfig>;
  agentDocker?: Partial<SandboxDockerConfig>;
}): SandboxDockerConfig {
  return {
    image: ... ?? DEFAULT_SANDBOX_IMAGE,
    // "openclaw-sandbox:bookworm-slim"
    containerPrefix: ... ?? DEFAULT_SANDBOX_CONTAINER_PREFIX,
    workdir: ... ?? DEFAULT_SANDBOX_WORKDIR,  // "/workspace"
    readOnlyRoot: ... ?? true,     // 只读根文件系统
    tmpfs: ... ?? ["/tmp", "/var/tmp", "/run"],  // 可写临时目录
    network: ... ?? "none",        // 默认无网络
    capDrop: ... ?? ["ALL"],       // 丢弃所有 Linux 能力
    env,                           // 环境变量（默认 LANG=C.UTF-8）
    // ... 资源限制：pidsLimit, memory, cpus 等
  };
}
```

默认的安全配置非常严格：

参数	默认值	安全含义
`readOnlyRoot`	`true`	容器根文件系统只读，防止恶意修改系统文件
`network`	`"none"`	无网络访问，防止数据泄露
`capDrop`	`["ALL"]`	丢弃所有 Linux 内核能力，以最低权限运行
`tmpfs`	`/tmp, /var/tmp, /run`	只有临时目录可写，重启后自动清除

> **衍生解释：Linux Capabilities**
> Linux Capabilities 是将传统的 root 超级权限拆分成多个细粒度权限的机制。例

> 如， `CAP_NET_ADMIN` 允许配置网络， `CAP_SYS_PTRACE` 允许调试进程。 `capDrop: ["ALL"]` 表示丢弃所有这些能力，容器内的进程即使以 root 身份运行，也几乎没有特权操作的能力。

浏览器沙箱

对于需要浏览器能力的 Agent，OpenClaw 提供了独立的浏览器沙箱：

```
export function resolveSandboxBrowserConfig(params) {
  return {
    enabled: ... ?? false,              // 默认关闭
    image: ... ?? DEFAULT_SANDBOX_BROWSER_IMAGE,
    // "openclaw-sandbox-browser:bookworm-slim"
    cdpPort: ... ?? 9222,               // Chrome DevTools Protocol 端口
    vncPort: ... ?? 5900,               // VNC 远程桌面端口
    noVncPort: ... ?? 6080,             // noVNC Web 端口
    headless: ... ?? false,             // 非无头模式（可通过 VNC 观察）
    enableNoVnc: ... ?? true,           // 启用 Web VNC 访问
    allowHostControl: ... ?? false,     // 禁止宿主浏览器控制
    autoStart: ... ?? true,             // 工具调用时自动启动
    autoStartTimeoutMs: ... ?? 12_000,  // 自动启动超时 12 秒
  };
}
```

浏览器沙箱使用专门的 Docker 镜像（ `openclaw-sandbox-browser:bookworm-slim` ），内置 Chromium 浏览器和 VNC 服务器。Agent 通过 CDP（Chrome DevTools Protocol）远程控制容器内的浏览器，而管理员可以通过 noVNC 的 Web 界面实时观察 Agent 的浏览器操作。

容器生命周期管理

沙箱容器的清理通过 `resolveSandboxPruneConfig()` 配置：

```
export function resolveSandboxPruneConfig(params) {
  return {
    idleHours: ... ?? DEFAULT_SANDBOX_IDLE_HOURS,    // 24 小时
    maxAgeDays: ... ?? DEFAULT_SANDBOX_MAX_AGE_DAYS,  // 7 天
  };
}
```

参数	默认值	含义
`idleHours`	24	容器空闲超过 24 小时后自动停止

参数	默认值	含义
`maxAgeDays`	7	容器存活超过 7 天后强制清除

这两个参数共同确保沙箱容器不会无限占用系统资源。

沙箱状态目录

沙箱的运行状态持久化在 `~/.openclaw/sandbox/` 目录下：

```
export const SANDBOX_STATE_DIR =
  path.join(resolvedSandboxStateDir, "sandbox");
export const SANDBOX_REGISTRY_PATH =
  path.join(SANDBOX_STATE_DIR, "containers.json");
export const SANDBOX_BROWSER_REGISTRY_PATH =
  path.join(SANDBOX_STATE_DIR, "browsers.json");
```

`containers.json` 记录所有活跃的沙箱容器信息（容器 ID、创建时间、关联的 Agent 等）， `browsers.json` 记录浏览器沙箱的对应关系。

完整的配置示例

以下展示一个多 Agent 沙箱配置的完整示例：

```
{
  "agents": {
    "list": [
      {
        "id": "main",
        "default": true,
        // 主 Agent 不使用沙箱
      },
      {
        "id": "coder",
        "sandbox": {
          "mode": "auto",
          "scope": "session",
          "workspaceAccess": "readwrite",
          "docker": {
            "network": "bridge",  // 需要网络（如 npm install）
            "memory": "2g",
            "cpus": 2
          }
        },
        "tools": {
          "sandbox": {
            "tools": {
              "allow": ["exec", "read", "write", "edit", "process"],
              "deny": ["sessions_spawn"]  // 禁止子 Agent 嵌套
            }
          }
        }
      },
      {
        "id": "browser-agent",
        "sandbox": {
          "mode": "always",
          "scope": "agent",
          "docker": {
            "network": "bridge"  // 浏览器需要网络
          },
          "browser": {
            "enabled": true,
            "headless": true
          }
        },
        "tools": {
```

```
          "sandbox": {
            "tools": {
              "allow": ["browser", "read", "image"],
              "deny": ["exec", "process", "write"]  // 只读 + 浏览器
            }
          }
        }
      }
    ]
  }
}
```

在这个配置中：

- **main** Agent 直接在宿主运行，拥有完整权限
- **coder** Agent 在 Docker 沙箱中运行，每个会话一个容器，有网络访问和 2GB 内存限制，但不能派生子 Agent
- **browser-agent** 在浏览器沙箱中运行，只能使用浏览器和文件读取工具，不能执行命令或写入文件

这种配置实现了**纵深防御**——即使 AI Agent 被恶意指令诱导，它能造成的损害也被严格限制在其沙箱边界内。

本节小结

1. **默认工具策略** 遵循"执行能力开放、通信能力封闭"原则：允许文件和进程操作，禁止浏览器、通道、定时任务等。
2. **三级优先级** 策略解析：Agent 级 > 全局 > 默认常量，每级都记录来源便于审计。
3. **黑名单优先** 的匹配算法：deny 优先于 allow，支持 `*` 通配符匹配。
4. **image 工具特殊保护**：除非显式禁止，否则自动加入白名单，保障多模态基础能力。
5. **沙箱容器安全默认值** 极为严格：只读根文件系统、无网络、丢弃所有 Linux 能力。
6. **三种容器范围**：`agent`（Agent 共享）、`session`（会话隔离）、`shared`（全局共享），平衡隔离度与资源消耗。
7. **浏览器沙箱** 使用独立镜像，通过 CDP 控制 + VNC 观察，实现浏览器操作的安全隔离。
8. **容器生命周期管理**：24 小时空闲停止 + 7 天强制清除，防止资源泄漏。